

ARES Protocol

Connection and operating guide for REST/JSON, XML, Zenoh, and ROS2DDS

[REST/JSON](#)
[XML](#)
[Zenoh](#)
[ROS2DDS](#)

This document describes how clients connect to an already-running ARES server. It documents the external protocol only: URLs, keys, payloads, and expected responses.

Contents

1	Connection points	2
2	Shared data rules	2
2.1	Resource identifiers	2
2.2	Common enum values	2
2.3	Error response	2
3	Operating story	2
3.1	Story overview	2
3.2	Full REST story: robot claims dock A	3
3.3	Contention story: second robot is denied until release	4
3.4	Shared-zone story: capacity	4
3.5	Full XML story: same claim flow	4
4	REST / JSON protocol	5
4.1	Health	5
4.2	Map lookup	6
4.3	Route planning	6
4.4	Robots	7
4.5	Claims	7
4.6	Leases and snapshot	8
5	XML protocol	8
5.1	XML endpoint table	8
5.2	XML calls	9
6	Zenoh and ROS2DDS	10
6.1	ROS2 service catalog	10
6.2	<code>ares_interfaces/srv/Json</code>	10
6.3	Full ROS2 JSON services	11
6.4	ROS2 connection story	12
6.5	ROS2 troubleshooting	13
6.6	Internal ROS2DDS response encoding	13
7	ARES native Zenoh services	13
7.1	Complete Zenoh service catalog	13
7.2	Native Zenoh operating sequence	16

1 Connection points

The operator running ARES should provide these values.

Name	Example	Used for
REST_BASE	<code>http://84.22.103.80:8080/ares/v1</code>	REST/JSON calls.
XML_BASE	<code>http://84.22.103.80:8080/ares/v1</code>	XML calls, if the XML adapter is exposed.
ZENO_ENDPOINT	<code>tcp/84.22.103.80:7447</code>	Zenoh bridge or peer connection.
ROS2_SERVICE	<code>/ares/v1/health</code>	ARES JSON service through <code>zenoh-bridge-ros2dds</code> .

Zenoh endpoint syntax: use `tcp/84.22.103.80:7447`. Do not use `tcp://84.22.103.80:7447`.

2 Shared data rules

2.1 Resource identifiers

Zones, nodes, and edges are UUID-based internally, but client-facing calls can use either a UUID or a numeric alias. Numeric aliases should be sent as strings to keep JSON and XML compatible.

```
"100"
```

```
"00000000-0000-0000-0000-000000000100"
```

2.2 Common enum values

Field	Allowed values
Claim target kind	<code>Zone</code> , <code>Node</code> , <code>Edge</code>
Claim access mode	<code>Shared</code> , <code>Exclusive</code>
Robot progress	<code>Idle</code> , <code>FollowingRoute</code> , <code>Waiting</code> , <code>Queued</code> , <code>Blocked</code> , <code>Replanning</code>
Lease disposition	<code>Active</code> , <code>Released</code> , <code>Expired</code> , <code>Revoked</code>

2.3 Error response

REST and XML API errors return HTTP `400`. JSON error body:

```
{"message": "what went wrong"}
```

3 Operating story

This is the normal ARES flow: check the server, read the map, register the robot, plan movement, ask for access to a zone, then keep the server updated as the robot moves.

3.1 Story overview

Step	Action	Why
1	Health check	Confirm that the ARES server is reachable.
2	Read map resources	Find zone, node, and edge IDs. XML does not expose map lookup yet, so use REST for this step.
3	Register robot	Tell ARES the robot exists and where it starts.

Step	Action	Why
4	Plan route	Ask ARES for a graph route between nodes.
5	Evaluate claim	Dry-run zone/node/edge access before committing.
6	Submit claim	Store the request if ARES grants it.
7	Move / heartbeat	Update robot progress while it moves.
8	Release / cleanup	Remove claims or release leases when finished.
9	Snapshot	Read robots, active claims, and leases in one response.

3.2 Full REST story: robot claims dock A

Set the base URL:

```
REST BASE=http://84.22.103.80:8080/ares/v1
```

1. Check health:

```
curl $REST BASE/health
```

2. Read zones and nodes. In the fixed example, zone 100 is dock_a, node 1001 is the dock entry, node 1003 is the other side of the map.

```
curl $REST BASE/zones
```

```
curl $REST BASE/nodes
```

3. Register robot **1** at node **1001**.

```
curl -X POST $REST_BASE/robots -H 'content-type: application/json' -d
{"robot_id":1,"mission_id":9001,"current_node_id":"00000000-0000-0000-0000-000000001001","progress_state":"Idle","upc
```

4. Plan a route from node 1001 to node 1003.

```
curl -X POST $REST_BASE/routes/plan -H 'content-type: application/json' -d '{"start node id":"1001","goal node id":"1003","use penalties":false}'
```

5. Dry-run an exclusive claim on zone 100.

```
curl -X POST $REST_BASE/claims/evaluate -H 'content-type: application/json' -d
{'id':10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start tick":10,"end tick":100},"targets":[{"kind":"Zone","resource id":"100"}]}
```

If the response contains `"decision": "Grant"`, submit the claim.

6. Submit/store the granted claim.

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
{'id':10,'robot_id':1,'mission_id':9001,'access_mode':'Exclusive','priority':10,'requested_at_tick':10,'window':
{'start_tick':10,'end_tick':100},'targets':[{'kind':'Zone','resource id':'100'}]}
```

7. Confirm the claim is active.

```
curl $REST BASE/claims
```

8. Update robot progress as it moves.

```
curl -X POST $REST_BASE/robots/1/heartbeat -H 'content-type: application/json' -d '{"current node id": "1002", "current edge id": null, "updated at tick": 70}'
```

9. Read the whole fleet state.

```
curl $REST_BASE/fleet/snapshot
```

10. When done, remove the active claim.

```
curl -X DELETE $REST_BASE/claims/10
```

3.3 Contention story: second robot is denied until release

Robot **2** tries to claim the same exclusive zone while robot **1** still owns claim **10**.

```
curl -X POST $REST_BASE/robots -H 'content-type: application/json' -d
'{"robot_id":2,"mission_id":9002,"current_node_id":"00000000-0000-0000-0000-000000001003","progress_state":"Idle","up
```

```
curl -X POST $REST_BASE/claims/evaluate -H 'content-type: application/json' -d
'{"id":11,"robot_id":2,"mission_id":9002,"access_mode":"Exclusive","priority":5,"requested_at_tick":11,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}'
```

Expected decision while claim **10** is active:

```
{"decision":"Deny", "...":"..."}
```

Release robot **1** claim:

```
curl -X DELETE $REST_BASE/claims/10
```

Now robot **2** can submit:

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":11,"robot_id":2,"mission_id":9002,"access_mode":"Exclusive","priority":5,"requested_at_tick":12,"window":
{"start_tick":12,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}'
```

3.4 Shared-zone story: capacity

For shared zones, more than one robot may be granted if the zone policy allows capacity. In the fixed example, zone **101** is shared.

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":20,"robot_id":1,"mission_id":9101,"access_mode":"Shared","priority":1,"requested_at_tick":20,"window":
{"start_tick":20,"end_tick":120},"targets":[{"kind":"Zone","resource_id":"101"}]}'
```

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":21,"robot_id":2,"mission_id":9102,"access_mode":"Shared","priority":1,"requested_at_tick":21,"window":
{"start_tick":20,"end_tick":120},"targets":[{"kind":"Zone","resource_id":"101"}]}'
```

If capacity is full, another overlapping claim is denied:

```
curl -X POST $REST_BASE/claims/evaluate -H 'content-type: application/json' -d
'{"id":22,"robot_id":3,"mission_id":9103,"access_mode":"Shared","priority":1,"requested_at_tick":22,"window":
{"start_tick":20,"end_tick":120},"targets":[{"kind":"Zone","resource_id":"101"}]}'
```

3.5 Full XML story: same claim flow

Set:

```
XML_BASE=http://84.22.103.80:8080/ares/v1
```

XML has the claim/robot/route/lease calls. For map lookup, use REST first because XML map lookup is not exposed yet.

1. Check health:

```
curl -H 'accept: application/xml' $XML_BASE/health
```

2. Register robot:

```
curl -X POST $XML_BASE/robots -H 'content-type: application/xml' -d '<RobotState><robot_id>1</robot_id><mission_id>9001</mission_id><current_node_id>00000000-0000-0000-0000-000000001001</current_node_id><progress_state>Idle</progress_state><updated_at_tick>1</updated_at_tick></RobotState>'
```

3. Plan route:

```
curl -X POST $XML_BASE/routes/plan -H 'content-type: application/xml' -d '<PlanRouteRequest><start_node_id>1001</start_node_id><goal_node_id>1003</goal_node_id><use_penalties>false</use_penalties></PlanRouteRequest>'
```

4. Dry-run claim:

```
curl -X POST $XML_BASE/claims/evaluate -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

5. Submit claim:

```
curl -X POST $XML_BASE/claims -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

6. Heartbeat:

```
curl -X POST $XML_BASE/robots/1/heartbeat -H 'content-type: application/xml' -d '<HeartbeatRequest><current_node_id>1002</current_node_id><updated_at_tick>70</updated_at_tick></HeartbeatRequest>'
```

7. Snapshot:

```
curl -H 'accept: application/xml' $XML_BASE/fleet/snapshot
```

8. Add a lease directly if the integration uses leases:

```
curl -X POST $XML_BASE/leases -H 'content-type: application/xml' -d '<Lease><id>501</id><claim_id>10</claim_id><robot_id>1</robot_id><access_mode>Exclusive</access_mode><targets><kind>Zone</kind><resource_id>00000000-0000-0000-0000-000000000100</resource_id></targets><granted_at_tick>12</granted_at_tick><expires_at_tick>200</expires_at_tick><disposition>Active</disposition><active>true</active></Lease>'
```

9. Release lease:

```
curl -X POST $XML_BASE/leases/release -H 'content-type: application/xml' -d '<ReleaseLeaseRequest><lease_id>501</lease_id><released_at_tick>50</released_at_tick></ReleaseLeaseRequest>'
```

4 REST / JSON protocol

Set the base URL:

```
REST_BASE=http://84.22.103.80:8080/ares/v1
```

All REST request bodies are JSON. All successful responses are JSON.

4.1 Health

Method	Path / Key	Request	Response
GET	/health	none	Health

```
curl $REST_BASE/health
```

```
{"status": "ok", "version": "0.0.2"}
```

4.2 Map lookup

Method	Path	Meaning
GET	/zones	List zones.
GET	/zones/{id}	Get zone by UUID or numeric alias.
GET	/nodes	List graph nodes.
GET	/nodes/{id}	Get node by UUID or numeric alias.
GET	/edges	List graph edges.
GET	/edges/{id}	Get edge by UUID or numeric alias.

```
curl $REST_BASE/zones
curl $REST_BASE/zones/100
curl $REST_BASE/nodes
curl $REST_BASE/nodes/1001
curl $REST_BASE/edges
curl $REST_BASE/edges/2001
```

Zone shape:

```
{
  "id": "00000000-0000-0000-0000-000000000100",
  "numeric_id": 100,
  "name": "dock_a",
  "kind": "zone",
  "parent_id": "00000000-0000-0000-0000-000000000001",
  "child_ids": [],
  "node_ids": [],
  "properties": {"traffic.policy": "exclusive"}
}
```

4.3 Route planning

Method	Path / Key	Request	Response
POST	/routes/plan	PlanRouteRequest	PlanRouteResponse

```
curl -X POST $REST_BASE/routes/plan -H 'content-type: application/json' -d
'{"start_node_id": "1001", "goal_node_id": "1003", "use_penalties": false}'
```

```
{
  "start_node_id": "1001",
  "goal_node_id": "1003",
  "use_penalties": false
}
```

Response fields:

found	Whether a route was found.
distance	Graph distance / cost.
plan	Route plan if found.
failure	Failure diagnostics if not found.

4.4 Robots

Method	Path	Meaning
GET	/robots	List registered robots.
POST	/robots	Register or replace robot state.
GET	/robots/{id}	Get robot state.
DELETE	/robots/{id}	Unregister robot.
POST	/robots/{id}/heartbeat	Update current node/edge and tick.
POST	/robots/{id}/route	Assign a route plan.
POST	/robots/{id}/schedule	Schedule from an existing claim.

Register:

```
curl -X POST $REST_BASE/robots -H 'content-type: application/json' -d
'{"robot_id":1,"mission_id":9001,"current_node_id":"00000000-0000-0000-0000-000000001001","progress_state":"Idle","up
```

Heartbeat:

```
curl -X POST $REST_BASE/robots/1/heartbeat -H 'content-type: application/json' -d
'{"current_node_id":"1002","current_edge_id":null,"updated_at_tick":70}'
```

Schedule:

```
curl -X POST $REST_BASE/robots/1/schedule -H 'content-type: application/json' -d
'{"claim_id":10,"start_tick":20,"ticks_per_cost_unit":1.0,"access_mode":"Exclusive"}'
```

4.5 Claims

Method	Path	Meaning
GET	/claims	List active claim requests.
POST	/claims/evaluate	Dry-run claim; does not store it.
POST	/claims	Evaluate and store only if granted.
GET	/claims/{id}	Get active claim request.
DELETE	/claims/{id}	Remove active claim request.

Submit claim:

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}}]'
```

Claim body:

```
{
  "id": 10,
  "robot_id": 1,
  "mission_id": 9001,
  "access_mode": "Exclusive",
  "priority": 10,
  "requested_at_tick": 10,
  "window": {"start_tick": 10, "end_tick": 100},
  "targets": [{"kind": "Zone", "resource_id": "100"}]}
}
```

4.6 Leases and snapshot

Method	Path	Meaning
GET	/leases	List leases.
POST	/leases	Add lease directly.
POST	/leases/release	Release lease by body.
DELETE	/leases/{id}	Release lease by path.
GET	/fleet/snapshot	Robots, active claim requests, and leases.

```
curl $REST_BASE/fleet/snapshot
```

5 XML protocol

Set the XML base URL:

```
XML_BASE=http://84.22.103.80:8080/ares/v1
```

XML uses the same /ares/v1 prefix as REST. The server selects XML when:

- a GET request sends Accept: application/xml
- a request with a body sends Content-Type: application/xml

Body requests return XML automatically when their body was XML.

5.1 XML endpoint table

Method	Path	Request root
GET	/health	none
GET	/fleet/snapshot	none
POST	/routes/plan	<PlanRouteRequest>
GET	/zones	none
GET	/zones/{id}	none
GET	/nodes	none
GET	/nodes/{id}	none
GET	/edges	none
GET	/edges/{id}	none
GET	/robots	none
POST	/robots	<RobotState>
GET	/robots/{id}	none
DELETE	/robots/{id}	none
POST	/robots/{id}/heartbeat	<HeartbeatRequest>
POST	/robots/{id}/route	<AssignRouteRequest>
POST	/robots/{id}/schedule	<ScheduleRobotRouteRequest>
GET	/claims	none
POST	/claims	<ClaimRequestWire>
POST	/claims/evaluate	<ClaimRequestWire>
GET	/claims/{id}	none
DELETE	/claims/{id}	none

Method	Path	Request root
GET	/leases	none
POST	/leases	<Lease>
POST	/leases/release	<ReleaseLeaseRequest>
DELETE	/leases/{id}	none

5.2 XML calls

Health:

```
curl -H 'accept: application/xml' $XML_BASE/health
```

Zones:

```
curl -H 'accept: application/xml' $XML_BASE/zones
```

Get zone:

```
curl -H 'accept: application/xml' $XML_BASE/zones/100
```

Nodes:

```
curl -H 'accept: application/xml' $XML_BASE/nodes
```

Edges:

```
curl -H 'accept: application/xml' $XML_BASE/edges
```

Route:

```
curl -X POST $XML_BASE/routes/plan -H 'content-type: application/xml' -d
'<PlanRouteRequest><start_node_id>1001</start_node_id><goal_node_id>1003</
goal_node_id><use_penalties>false</use_penalties></PlanRouteRequest>'
```

Register robot:

```
curl -X POST $XML_BASE/robots -H 'content-type: application/xml' -d '<RobotState><robot_id>1</
robot_id><mission_id>9001</mission_id><current_node_id>00000000-0000-0000-0000-000000001001</
current_node_id><progress_state>Idle</progress_state><updated_at_tick>1</updated_at_tick></RobotState>'
```

Heartbeat:

```
curl -X POST $XML_BASE/robots/1/heartbeat -H 'content-type: application/xml' -d
'<HeartbeatRequest><current_node_id>1002</current_node_id><updated_at_tick>70</updated_at_tick></
HeartbeatRequest>'
```

Claim evaluate:

```
curl -X POST $XML_BASE/claims/evaluate -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</
id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</
priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</
end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

Submit claim:

```
curl -X POST $XML_BASE/claims -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</
id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</
priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</
end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

List claims:

```
curl -H 'accept: application/xml' $XML_BASE/claims
```

Lease release:

```
curl -X POST $XML_BASE/leases/release -H 'content-type: application/xml' -d
'<ReleaseLeaseRequest><lease_id>501</lease_id><released_at_tick>50</released_at_tick></
ReleaseLeaseRequest>'
```

6 Zenoh and ROS2DDS

The ARES server stays pure Zenoh. It does not link ROS2. ROS2 machines call it through `zenoh-bridge-ros2dds`, which translates a ROS2 service request into a Zenoh query.

6.1 ROS2 service catalog

This is the complete ROS2DDS service surface currently exposed by ARES.

ROS2 service	ROS2 type	Zenoh key	Meaning
<code>/ares/v1/health</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/health</code>	Health/version as JSON.
<code>/ares/v1/fleet/snapshot</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/fleet/snapshot</code>	Robots, claims, and leases.
<code>/ares/v1/routes/plan</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/routes/plan</code>	Plan route from JSON request.
<code>/ares/v1/robots/register</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/robots/register</code>	Register robot from JSON request.
<code>/ares/v1/robots/list</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/robots/list</code>	List robots.
<code>/ares/v1/robots/heartbeat</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/robots/heartbeat</code>	Update robot progress.
<code>/ares/v1/robots/assign_route</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/robots/assign_route</code>	Assign route plan.
<code>/ares/v1/robots/schedule</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/robots/schedule</code>	Schedule robot from claim.
<code>/ares/v1/claims/list</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/claims/list</code>	List active claims.
<code>/ares/v1/claims/evaluate</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/claims/evaluate</code>	Dry-run claim.
<code>/ares/v1/claims/request</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/claims/request</code>	Submit claim if granted.
<code>/ares/v1/leases/list</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/leases/list</code>	List leases.
<code>/ares/v1/leases/add</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/leases/add</code>	Add lease.
<code>/ares/v1/leases/release</code>	<code>ares_interfaces/srv/Json</code>	<code>ares/v1/leases/release</code>	Release lease.

The `/ares/v1/...` services are the real ARES ROS2DDS service surface.

6.2 `ares_interfaces/srv/Json`

The full ARES ROS2 service surface uses one generic JSON service type:

```
string request
---
```

```
bool success
string response
```

The `.srv` file is included in this repository at:

```
ros/ares_interfaces/srv/Json.srv
```

Rules:

<code>request</code>	JSON request object encoded as a string. Use <code>{}</code> for no-input services.
<code>success</code>	<code>true</code> when the call succeeded; <code>false</code> when ARES returned an error.
<code>response</code>	On success: JSON response encoded as a string. On error: error text.

Example call:

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
```

6.3 Full ROS2 JSON services

Health:

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
```

Fleet snapshot:

```
ros2 service call /ares/v1/fleet/snapshot ares_interfaces/srv/Json "{request: '{}'}"
```

Route planning:

```
ros2 service call /ares/v1/routes/plan ares_interfaces/srv/Json "{request: '{\"start_node_id\": \"1001\", \"goal_node_id\": \"1003\", \"use_penalties\": false}'}"
```

Register robot:

```
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json "{request: '{\"robot_id\": 1, \"mission_id\": 9001, \"current_node_id\": \"00000000-0000-0000-0000-000000001001\", \"progress_state\": \"Idle\", \"updated_at_tick\": 1}'}"
```

List robots:

```
ros2 service call /ares/v1/robots/list ares_interfaces/srv/Json "{request: '{}'}"
```

Heartbeat:

```
ros2 service call /ares/v1/robots/heartbeat ares_interfaces/srv/Json "{request: '{\"robot_id\": 1, \"heartbeat\": {\"current_node_id\": \"1002\", \"current_edge_id\": null, \"updated_at_tick\": 70}'}"
```

Assign route:

```
ros2 service call /ares/v1/robots/assign_route ares_interfaces/srv/Json "{request: '{\"robot_id\": 1, \"assignment\": {\"route_plan\": {\"start_node_id\": \"00000000-0000-0000-0000-000000001001\", \"goal_node_id\": \"00000000-0000-0000-0000-000000001003\", \"steps\": [], \"traversed_node_ids\": [], \"traversed_edge_ids\": [], \"traversed_zone_ids\": [], \"traversed_node_zone_ids\": [], \"traversed_edge_zone_ids\": [], \"total_cost\": 0.0}, \"horizon\": 100, \"updated_at_tick\": 10}'}"
```

Schedule:

```
ros2 service call /ares/v1/robots/schedule ares_interfaces/srv/Json "{request: '{\"robot_id\": 1, \"schedule\": {\"claim_id\": 10, \"start_tick\": 20, \"ticks_per_cost_unit\": 1.0, \"access_mode\": \"Exclusive\"}'}"
```

List claims:

```
ros2 service call /ares/v1/claims/list ares_interfaces/srv/Json "{request: '{}'}"
```

Evaluate claim:

```
ros2 service call /ares/v1/claims/evaluate ares_interfaces/srv/Json "{request: '{\"id\":10,
\"robot_id\":1,\"mission_id\":9001,\"access_mode\":\"Exclusive\", \"priority\":10,
\"requested_at_tick\":10,\"window\":{\"start_tick\":10,\"end_tick\":100},\"targets\":[{\"kind\":\"Zone\",
\"resource_id\":\"100\"}]}'}"
```

Submit claim:

```
ros2 service call /ares/v1/claims/request ares_interfaces/srv/Json "{request: '{\"id\":10,\"robot_id\":1,
\"mission_id\":9001,\"access_mode\":\"Exclusive\", \"priority\":10,\"requested_at_tick\":10,\"window\":
{\"start_tick\":10,\"end_tick\":100},\"targets\":[{\"kind\":\"Zone\", \"resource_id\":\"100\"}]}'}"
```

List leases:

```
ros2 service call /ares/v1/leases/list ares_interfaces/srv/Json "{request: '{}'}"
```

Add lease:

```
ros2 service call /ares/v1/leases/add ares_interfaces/srv/Json "{request: '{\"id\":501,\"claim_id\":10,
\"robot_id\":1,\"access_mode\":\"Exclusive\", \"targets\":[{\"kind\":\"Zone\", \"resource_id\":
\"00000000-0000-0000-0000-000000000100\"}],\"granted_at_tick\":12,\"expires_at_tick\":200,
\"disposition\":\"Active\", \"active\":true}'}"
```

Release lease:

```
ros2 service call /ares/v1/leases/release ares_interfaces/srv/Json "{request: '{\"lease_id\":501,
\"released_at_tick\":50}'}"
```

6.4 ROS2 connection story

On the ARES machine, the server listens for Zenoh peers on `tcp/0.0.0.0:7447`. On the ROS2 machine, connect the bridge to the ARES machine:

```
zenoh-bridge-ros2dds -e tcp/TIMENAV_IP:7447
```

Use `tcp/TIMENAV_IP:7447`, not `tcp://TIMENAV_IP:7447`.

In another ROS2 terminal, source ROS and call the service:

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
```

Expected CLI shape:

```
requester: making request: ares_interfaces.srv.Json_Request(request='{}')
response:
ares_interfaces.srv.Json_Response(
  success=true,
  response='{\"status\":\"ok\", \"version\":\"0.0.2\"}'
)
```

If the bridge logs a route like this, the ROS2-to-Zenoh path is working:

```
Route Service Client (ROS:/ares/v1/health <-> Zenoh:ares/v1/health) created
```

`ros2 service list` may not show the ARES services. That is expected in the pure-Zenoh setup. The call works because the bridge creates the route dynamically when the request is made.

6.5 ROS2 troubleshooting

Symptom	Fix / explanation
Unicast not supported for tcp: protocol	The endpoint used <code>tcp://...</code> . Use <code>tcp/...</code> instead.
<code>ros2 service list</code> does not show the service	Expected for this pure-Zenoh setup. Call the <code>/ares/v1/...</code> service directly.
waiting for service to become available...	Keep the bridge running, then call directly. The bridge creates the route when it sees the client.
received invalid request ... less than 20 bytes	Use CycloneDDS on the ROS2 side: <code>export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp</code> .
No response	Check firewall/NAT and confirm the bridge can reach <code>tcp/TIMENAV_IP:7447</code> .

6.6 Internal ROS2DDS response encoding

ARES replies with a CDR-encoded `ares_interfaces/srv/Json_Response`:

```

00 01 00 00      CDR little-endian header
01              bool success
00 00 00        padding to 4 bytes
<u32 len including NUL> string length
<UTF-8 JSON bytes>
00              trailing NUL
<padding to 4 bytes>
```

7 ARES native Zenoh services

These are the ARES native Zenoh services from the `robo` adapter. They are JSON queryables under the `ares/v1` prefix. Request payloads are JSON strings where the table says a payload is required. Replies are JSON. Errors are Zenoh error replies with `{"message": "..."}.`

7.1 Complete Zenoh service catalog

Key	Payload	Response
<code>ares/v1/health</code>	none	<code>Health</code>
<code>ares/v1/fleet/snapshot</code>	none	<code>FleetSnapshot</code>
<code>ares/v1/routes/plan</code>	<code>PlanRouteRequest</code> JSON	<code>PlanRouteResponse</code>
<code>ares/v1/robots/register</code>	<code>RobotState</code> JSON	<code>RobotState</code>
<code>ares/v1/robots/list</code>	none	<code>RobotState[]</code>
<code>ares/v1/robots/heartbeat</code>	<code>{"robot_id":1,"heartbeat":...}</code>	<code>RobotState</code>
<code>ares/v1/robots/assign_route</code>	<code>{"robot_id":1,"assignment":...}</code>	<code>RobotState</code>
<code>ares/v1/robots/schedule</code>	<code>{"robot_id":1,"schedule":...}</code>	<code>ScheduleDecision</code>
<code>ares/v1/claims/list</code>	none	<code>ClaimRequest[]</code>
<code>ares/v1/claims/evaluate</code>	<code>ClaimRequestWire</code> JSON	<code>ClaimEvaluation</code>
<code>ares/v1/claims/request</code>	<code>ClaimRequestWire</code> JSON	<code>ClaimEvaluation</code>
<code>ares/v1/leases/list</code>	none	<code>Lease[]</code>
<code>ares/v1/leases/add</code>	<code>Lease</code> JSON	<code>Lease</code>
<code>ares/v1/leases/release</code>	<code>ReleaseLeaseRequest</code> JSON	<code>bool</code>

7.1.1 `ares/v1/health`

Checks that the ARES service is alive.

Payload	none
Response	Health: {"status":"ok","version":"0.0.2"}

7.1.2 ares/v1/fleet/snapshot

Returns the whole live coordination state: robots, active claim requests, and leases.

Payload	none
Response	FleetSnapshot: {"robots":[],"requests":[],"leases":[]}

7.1.3 ares/v1/routes/plan

Plans a route between two graph nodes.

Payload	PlanRouteRequest JSON
Response	PlanRouteResponse JSON

```
{"start_node_id":"1001","goal_node_id":"1003","use_penalties":false}
```

7.1.4 ares/v1/robots/register

Registers or replaces a robot state.

Payload	RobotState JSON
Response	RobotState JSON

```
{"robot_id":1,"mission_id":9001,"current_node_id":"00000000-0000-0000-0000-000000001001","progress_state":"Idle","update_at_tick":70}
```

7.1.5 ares/v1/robots/list

Lists all registered robots.

Payload	none
Response	RobotState[] JSON

7.1.6 ares/v1/robots/heartbeat

Updates the robot's current node/edge and timestamp.

Payload	RobotHeartbeatEnvelope JSON
Response	RobotState JSON

```
{"robot_id":1,"heartbeat":{"current_node_id":"1002","current_edge_id":null,"updated_at_tick":70}}
```

7.1.7 ares/v1/robots/assign_route

Assigns an already computed route plan to a robot.

Payload	RobotAssignRouteEnvelope JSON
Response	RobotState JSON

```
{
  "robot_id": 1,
  "assignment": {
    "route_plan": {
      "start_node_id": "00000000-0000-0000-0000-000000001001",
      "goal_node_id": "00000000-0000-0000-0000-000000001003",

```

```

    "steps": [],
    "traversed_node_ids": [],
    "traversed_edge_ids": [],
    "traversed_zone_ids": [],
    "traversed_node_zone_ids": [],
    "traversed_edge_zone_ids": [],
    "total_cost": 0.0
  },
  "horizon": 100,
  "updated_at_tick": 10
}

```

7.1.8 ares/v1/robots/schedule

Schedules a robot from an existing claim.

Payload	RobotScheduleEnvelope JSON
Response	ScheduleDecision JSON

```

{"robot_id":1,"schedule":
{"claim_id":10,"start_tick":20,"ticks_per_cost_unit":1.0,"access_mode":"Exclusive"}}

```

7.1.9 ares/v1/claims/list

Lists active claim requests.

Payload	none
Response	ClaimRequest[] JSON

7.1.10 ares/v1/claims/evaluate

Dry-runs a claim without storing it.

Payload	ClaimRequestWire JSON
Response	ClaimEvaluation JSON

```

{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}

```

7.1.11 ares/v1/claims/request

Evaluates a claim and stores it if the decision is `Grant`.

Payload	ClaimRequestWire JSON
Response	ClaimEvaluation JSON

```

{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}

```

7.1.12 ares/v1/leases/list

Lists leases.

Payload	none
Response	Lease[] JSON

7.1.13 ares/v1/leases/add

Adds a lease directly.

Payload	Lease JSON
Response	Lease JSON

```
{
  "id": 501,
  "claim_id": 10,
  "robot_id": 1,
  "access_mode": "Exclusive",
  "targets": [
    {
      "kind": "Zone",
      "resource_id": "00000000-0000-0000-0000-000000000100"
    }
  ],
  "granted_at_tick": 12,
  "expires_at_tick": 200,
  "dis"
```

7.1.14 ares/v1/leases/release

Releases a lease.

Payload	ReleaseLeaseRequest JSON
Response	bool

```
{
  "lease_id": 501,
  "released_at_tick": 50
}
```

7.2 Native Zenoh operating sequence

The native Zenoh story mirrors REST:

Step	Zenoh key	Payload
1	ares/v1/health	none
2	ares/v1/robots/register	RobotState
3	ares/v1/routes/plan	PlanRouteRequest
4	ares/v1/claims/evaluate	ClaimRequestWire dry-run
5	ares/v1/claims/request	ClaimRequestWire store-if-granted
6	ares/v1/robots/heartbeat	RobotHeartbeatEnvelope
7	ares/v1/fleet/snapshot	none
8	ares/v1/leases/release	ReleaseLeaseRequest, if leases are used